CSA4704-DEEP LEARNING

NAME:HARITHI S **REG NO:192421151**

Experiment 28: Deep Boltzmann Machine / Convolutional Boltzmann Machine Simulation

Problem Statement: A textile company wants to model and generate fabric texture patterns. Spatial structure matters, so a fully connected RBM is not enough. Small grayscale texture patches or MNIST images. The objective is to simulate a two-layer DBM or a convolutional RBM and study its representations. Use NumPy, SciPy, TensorFlow, Keras, Matplotlib.

Tasks:

1. Build a two-hidden-layer DBM with mean-field inference, or a convolutional RBM with shared filters.

2. Train with approximate learning (mean-field plus Gibbs sampling).

3. Visualise filters and hidden-unit activations.

4. Generate samples and reconstructions.

5. Compare with the single-layer RBM from Experiment 25 on reconstruction quality.

FORMULA:

1. Energy Function of DBM

$E(v, h_1, h_2) = -v^T W_1 h_1 - h_1^T W_2 h_2 - a^T v - b_1^T h_1 - b_2^T h_2$

2. Hidden Layer 1 Activation

$P(h_1 = 1 \mid v, h_2) = \sigma(W_1^T v + W_2 h_2 + b_1)$

3. Hidden Layer 2 Activation

$P(h_2 = 1 \mid h_1) = \sigma(W_2^T h_1 + b_2)$

4. Visible Layer Reconstruction

$P(v = 1 \mid h_1) = \sigma(W_1 h_1 + a)$

5. Sigmoid Activation Function

$\sigma(x) = 1 / (1 + e^{-x})$

6. Mean-Field Inference

$\mu_1 = \sigma(W_1^T v + W_2 \mu_2 + b_1)$

$\mu_2 = \sigma(W_2^T \mu_1 + b_2)$

7. Gibbs Sampling

$h_1 \sim \text{Bernoulli}(\sigma(W_1^T v + W_2 h_2 + b_1))$

$h_2 \sim \text{Bernoulli}(\sigma(W_2^T h_1 + b_2))$

$v \sim \text{Bernoulli}(\sigma(W_1 h_1 + a))$

8. DBM Weight Update

$W_1(\text{new}) = W_1(\text{old}) + \eta \times (\langle v h_1^T \rangle \text{data} - \langle v h_1^T \rangle \text{model})$

$W_2(\text{new}) = W_2(\text{old}) + \eta \times (\langle h_1 h_2^T \rangle \text{data} - \langle h_1 h_2^T \rangle \text{model})$

9. Convolution Operation

Feature Map = Input * Filter + Bias

10. Convolutional RBM Hidden Activation

$P(h = 1 \mid v) = \sigma(K * v + b)$

11. Convolutional RBM Visible Reconstruction

$\hat{v} = \sigma(\Sigma(K^T * h) + a)$

12. Reconstruction Error (MSE)

$MSE = (1/N) \times \Sigma_i (v_i - \hat{v}_i)^2$

13. Binary Cross-Entropy Loss

$BCE = -(1/N) \times \Sigma_i [v_i \log(\hat{v}_i) + (1 - v_i) \log(1 - \hat{v}_i)]$

14. Reconstruction Quality

Lower MSE = Better Reconstruction Quality

CODE:

```
import numpy as np

import matplotlib.pyplot as plt

from scipy.special import expit

from tensorflow import keras

np.random.seed(42)

(x_train, _), (x_test, _) = keras.datasets.mnist.load_data()

X = (x_train[:2000].reshape(-1, 784) / 255 > 0.5).astype("float32")

X_test = (x_test[:200].reshape(-1, 784) / 255 > 0.5).astype("float32")

W1 = np.random.normal(0, 0.01, (784, 256)).astype("float32")

W2 = np.random.normal(0, 0.01, (256, 128)).astype("float32")

a = np.zeros(784)

b1 = np.zeros(256)
```

```
b2 = np.zeros(128)

lr = 0.01

epochs = 5

batch_size = 64

# Mean-field inference

def mean_field(v):

    h1 = np.zeros((len(v), 256))

    h2 = np.zeros((len(v), 128))



    for _ in range(5):

        h1 = expit(a*0 + b1 + v @ W1 + h2 @ W2.T)

        h2 = expit(b2 + h1 @ W2)

    return h1, h2

# Gibbs sampling

def gibbs(v, steps=1):

    for _ in range(steps):

        h1_prob, h2_prob = mean_field(v)

        h1 = (np.random.rand(*h1_prob.shape) < h1_prob).astype(float)

        h2 = (np.random.rand(*h2_prob.shape) < h2_prob).astype(float)

        v_prob = expit(a + h1 @ W1.T)
```

```
        v = (np.random.rand(*v_prob.shape) < v_prob).astype(float)
    return v
# Train DBM
def train_dbm():
    global W1, W2, a, b1, b2
    errors = []
    fantasy_v = (np.random.rand(batch_size, 784) > 0.5).astype(float)

    for epoch in range(epochs):
        np.random.shuffle(X)
        total_error = 0
        batches = 0
        for i in range(0, len(X), batch_size):
            v0 = X[i:i + batch_size]
            size = len(v0)
            # Positive phase
            ph1, ph2 = mean_field(v0)
            # Negative phase
            if size == batch_size:
                vk = fantasy_v.copy()
```

```
else:
    vk = (np.random.rand(size, 784) > 0.5).astype(float)
vk = gibbs(vk, 1)
nh1, nh2 = mean_field(vk)
# Weight updates
W1 += lr * (
    v0.T @ ph1 - vk.T @ nh1
) / size

W2 += lr * (
    ph1.T @ ph2 - nh1.T @ nh2
) / size
a += lr * np.mean(v0 - vk, axis=0)
b1 += lr * np.mean(ph1 - nh1, axis=0)
b2 += lr * np.mean(ph2 - nh2, axis=0)
if size == batch_size:
    fantasy_v = vk.copy()
# Reconstruction error
total_error += np.mean(
    (v0 - expit(a + ph1 @ W1.T)) ** 2
```

```
            )
            batches += 1
        errors.append(total_error / batches)
        print("DBM Epoch:", epoch + 1, "Error:", round(errors[-1], 4))
    return errors
print("Training DBM")
dbm_errors = train_dbm()



# Visualize learned filters
plt.figure(figsize=(10, 10))
for i in range(100):
    plt.subplot(10, 10, i + 1)
    plt.imshow(W1[:, i].reshape(28, 28), cmap="gray")
    plt.axis("off")
plt.suptitle("DBM Learned Filters")
plt.tight_layout()
plt.show()
# Hidden-unit activations
h1, h2 = mean_field(X_test[:10])
```

```
plt.figure(figsize=(10, 5))
for i in range(5):
    plt.subplot(2, 5, i + 1)
    plt.imshow(h1[i].reshape(16, 16), cmap="gray")
    plt.title("Hidden 1")
    plt.axis("off")
    plt.subplot(2, 5, i + 6)
    plt.imshow(h2[i].reshape(8, 16), cmap="gray")
    plt.title("Hidden 2")
    plt.axis("off")
plt.tight_layout()
plt.show()
# Generate samples
sample = (np.random.rand(5, 784) > 0.5).astype(float)
generated = gibbs(sample, 100)
plt.figure(figsize=(10, 4))
for i in range(5):
    plt.subplot(1, 5, i + 1)
    plt.imshow(generated[i].reshape(28, 28), cmap="gray")
    plt.axis("off")
```

```
plt.suptitle("DBM Generated Samples")
plt.show()
# Reconstruct test images
h1, h2 = mean_field(X_test[:5])
reconstructed = expit(a + h1 @ W1.T)
plt.figure(figsize=(10, 4))
for i in range(5):
    plt.subplot(2, 5, i + 1)
    plt.imshow(X_test[i].reshape(28, 28), cmap="gray")
    plt.title("Original")
    plt.axis("off")
    plt.subplot(2, 5, i + 6)
    plt.imshow(reconstructed[i].reshape(28, 28), cmap="gray")
    plt.title("DBM")
    plt.axis("off")
plt.tight_layout()
plt.show()
# Train single-layer RBM for comparison
Wr = np.random.normal(0, 0.01, (784, 256))
ar = np.zeros(784)
```

```
br = np.zeros(256)

rbm_errors = []

for epoch in range(epochs):

    np.random.shuffle(X)

    total_error = 0

    batches = 0

    for i in range(0, len(X), batch_size):

        v0 = X[i:i + batch_size]



        ph0 = expit(br + v0 @ Wr)

        h0 = (np.random.rand(*ph0.shape) < ph0).astype(float)

        pv1 = expit(ar + h0 @ Wr.T)

        v1 = (np.random.rand(*pv1.shape) < pv1).astype(float)

        ph1 = expit(br + v1 @ Wr)

        Wr += lr * (v0.T @ ph0 - v1.T @ ph1) / len(v0)

        ar += lr * np.mean(v0 - v1, axis=0)

        br += lr * np.mean(ph0 - ph1, axis=0)

        total_error += np.mean((v0 - pv1) ** 2)

        batches +=

    rbm_errors.append(total_error / batches)
```

```
    print("RBM Epoch:", epoch + 1, "Error:", round(rbm_errors[-1], 4))

# Compare reconstruction errors

plt.figure(figsize=(8, 5))

plt.plot(dbm_errors, label="DBM")

plt.plot(rbm_errors, label="RBM")

plt.title("DBM vs RBM Reconstruction Error")

plt.xlabel("Epochs")

plt.ylabel("Reconstruction Error")

plt.legend()

plt.grid()

plt.show()

print("Final DBM Error:", round(dbm_errors[-1], 4))

print("Final RBM Error:", round(rbm_errors[-1], 4))
```

OUTPUT:

```
Training DBM

DBM Epoch: 1 Error: 0.1007

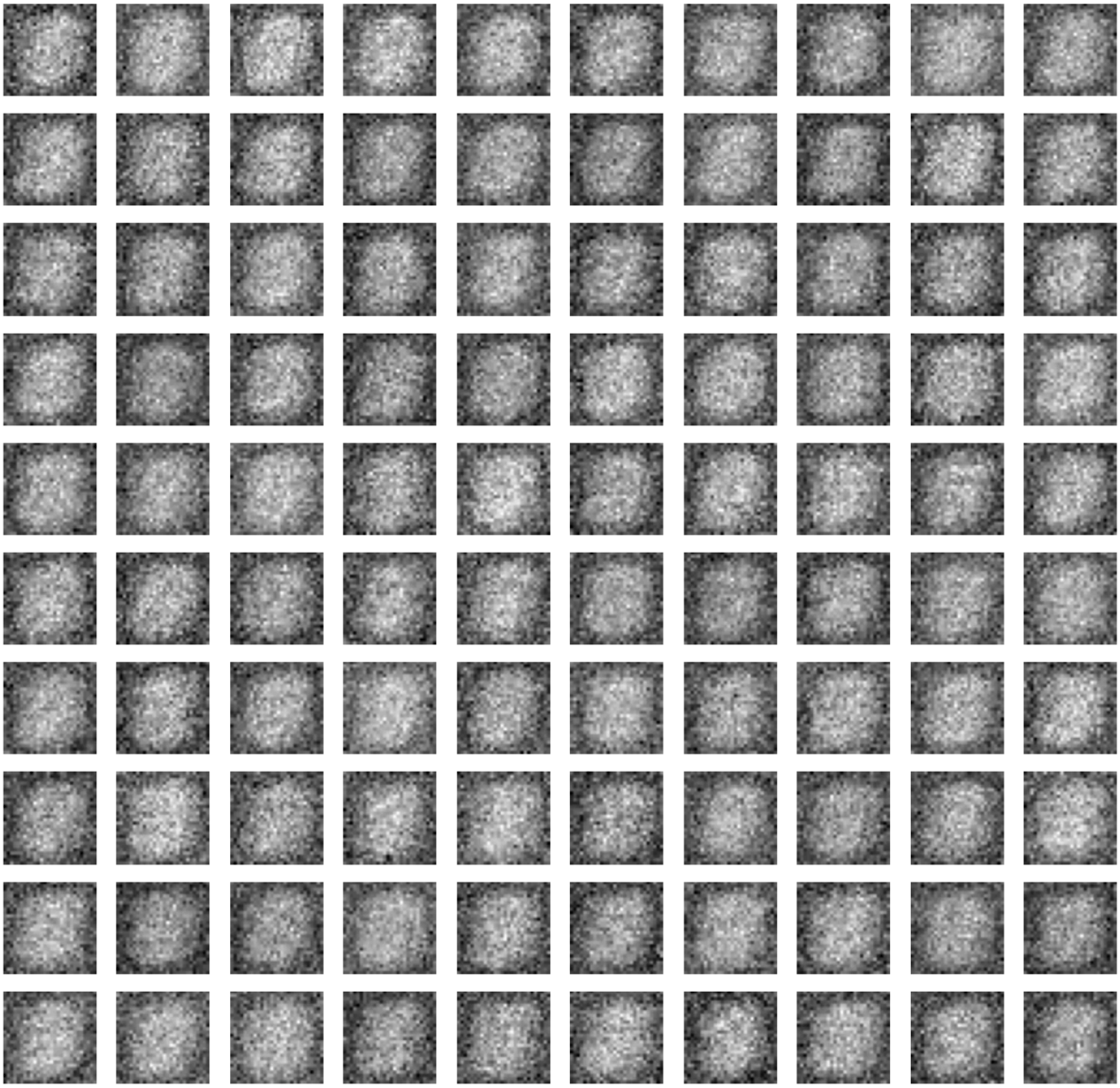
DBM Epoch: 2 Error: 0.0847

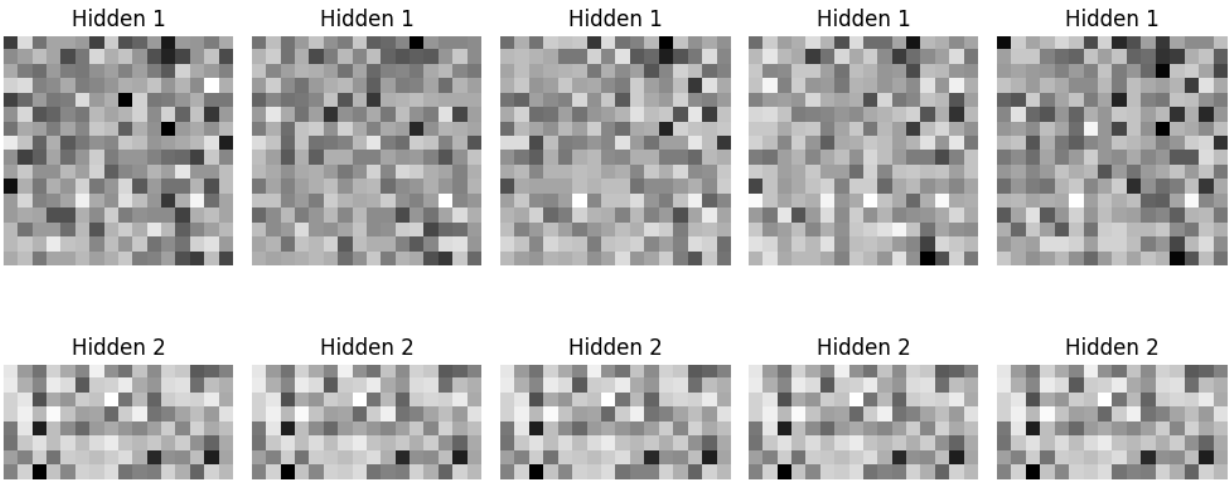
DBM Epoch: 3 Error: 0.0847

DBM Epoch: 4 Error: 0.0843

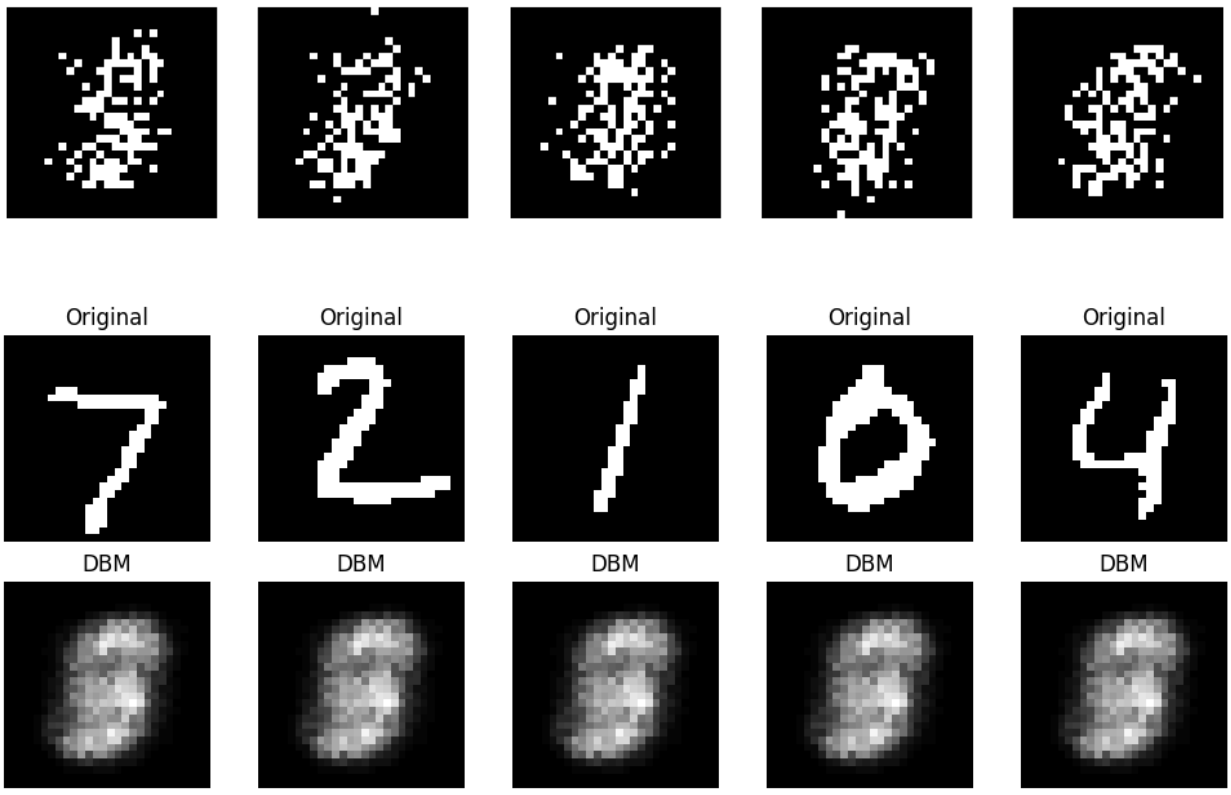
DBM Epoch: 5 Error: 0.0847
```

DBM Learned Filters





DBM Generated Samples



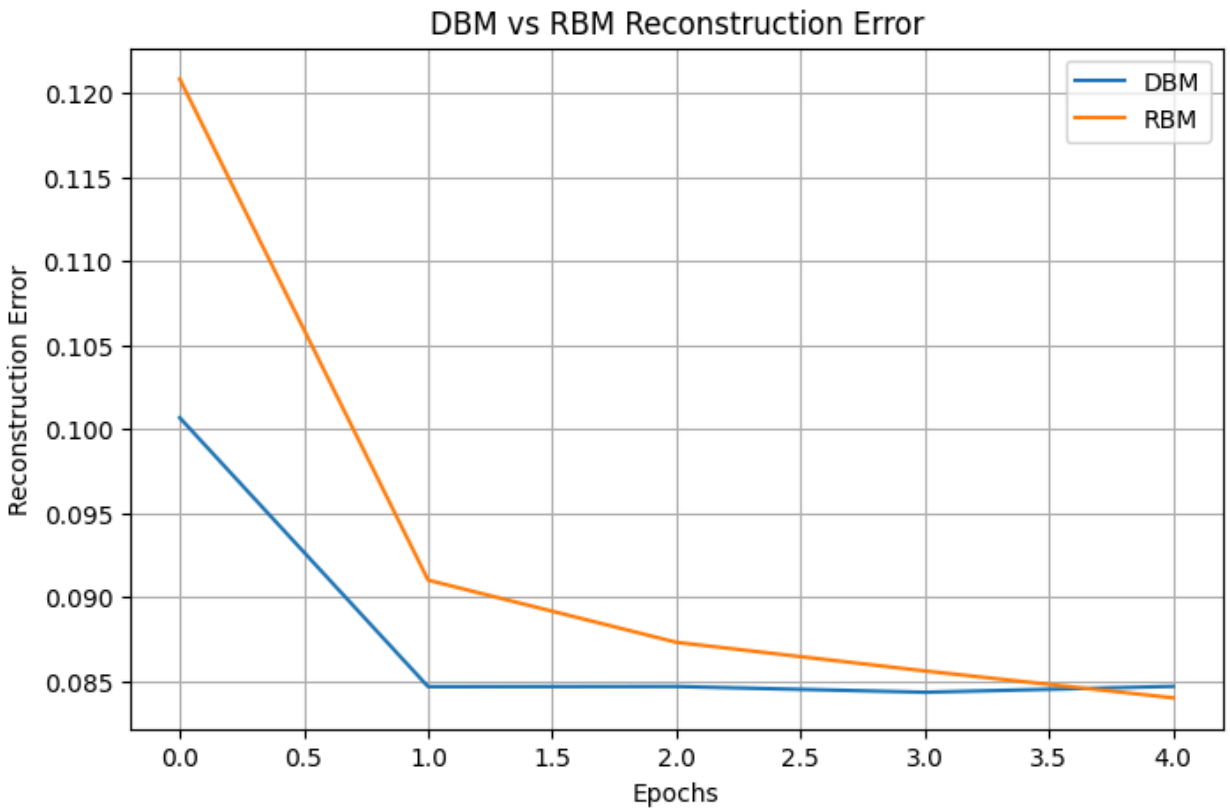
RBM Epoch: 1 Error: 0.1209

RBM Epoch: 2 Error: 0.091

RBM Epoch: 3 Error: 0.0873

RBM Epoch: 4 Error: 0.0856

RBM Epoch: 5 Error: 0.084



Final DBM Error: 0.0847

Final RBM Error: 0.084